Como conectar a ORACLE con Java

¿Que es JDBC y para que se utiliza?

JDBC es la interfaz que proporciona Java para la conexión a bases de datos. Son un conjunto de clases e interfaces que permiten a Java ejecutar consultas y ordenes en una base de datos.

Para poder conectar a Java con ORACLE lo primero que necesitamos es la implementación de JDBC para ORACLE. Esta implementación se encuentra en el archivo **Classes12.jar** que podemos encontrar en el siguiente directorio del servidor de Oracle.

%ORACLE_HOME%\jdbc\lib

ORACLE_HOME es una variable de entorno que se crea durante la instalación del servidor ORACLE, y que indica donde está instalado fisicamente ORACLE dentro del sistema de archivos, normalmente C:\oracle\ora92 para servidores Windows y ORACLE 9i. Esta .misma nomenclatura se aplica al hablar de la instalación cliente

Debemos configurar correctamente nuestra variable **CLASSPATH** para incluir en ella el archivo **Classes12.jar.**

Oracle proporciona los siguientes drivers en su implementacion JDBC:

- **JDBC Thin Driver**: No requiere de la instalación cliente de ORACLE. Ideal para Applets.
- **JDBC OCI** :API nativa de ORACLE, para aplicaciones J2EE.
- **JDBC KPRB driver**: (default connection) para Java Stored Procedures y Database JSP's.

Todos los drivers soportan la misma sintaxis y funcionalidad, por lo que nuestro código variará únicamente en la forma de crear la conexion.

Conectar a ORACLE con JDBC Thin Driver

El driver JDBC Thin se conecta y comunica con ORACLE a nivel de sockets. Proporciona su propia versión del protocolo Net8 (SQL*Net) sobre TCP/IP, por lo que no es necesario que exista una instalación de la parte cliente de ORACLE. Dado que este driver es código 100% Java es completamente independiente de la plataforma de ejecución, y como implementa su propia versión del protocolo es ideal para aplicaciones que se ejecutan sobre un navegador Web (applets)

```
import java.sql.*;
class dbAccess {
 public static void main (String args []) throws SQLException
 {

   DriverManager.registerDriver (new oracle.jdbc.driver.OracleDriver());
   Connection conn = DriverManager.getConnection
```

```
        ("jdbc:oracle:thin:@WIN01:1521:oracleBD", "user", "passw");
       // driver@machineName:port:SID           ,  userid,  password
   Statement stmt = conn.createStatement();
   ResultSet rset =
             stmt.executeQuery("select BANNER from SYS.V_$VERSION");
   while (rset.next())
        System.out.println (rset.getString(1));   // Print col 1
   stmt.close();


 }
}
```

Conectar a ORACLE con JDBC OCI Driver

El driver JDBC OCI usa el API ORACLE OCI (Oracle Call Interface) para interactuar con ORACLE, por lo que es necesaria la instalación y configuración de la parte cliente de ORACLE. El driver OCI trabaja con SQL*Net o Net8, por lo que tamdebemos haber creado un alias en el archivo TNS names (%ORACLE_HOME%\network\ADMIN):

```
ORACLEBD =
  (DESCRIPTION =
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST = WIN01)(PORT = 1521))
    )
    (CONNECT_DATA =
      (SERVICE_NAME = ORACLEBD)
    )
  )
```

El driver JDBC OCI permite realizar llamadas a ORACLE OCI directamente desde Java proporcionando
un alto grado de compatibilidad con una versión específica de ORACLE utilizando métodos nativos, pero específicos de la plataforma.

```
import java.sql.*;
class dbAccess
{
 public static void main (String args []) throws SQLException
 {
   try
   {
    Class.forName ("oracle.jdbc.driver.OracleDriver");
   }
   catch (ClassNotFoundException e)
   {
    e.printStackTrace();
   }
   Connection conn = DriverManager.getConnection
        ("jdbc:oracle:oci:@ORACLEBD", "user", "passw");
         //               @TNSNames_Entry,  userid,  password
   Statement stmt = conn.createStatement();
   ResultSet rset =
             stmt.executeQuery("select BANNER from SYS.V_$VERSION");
```

```
    while (rset.next())
          System.out.println (rset.getString(1));   // Print col 1
    stmt.close();
  }
 }
```

Conectar a ORACLE con JDBC KPRB Driver

Por último el driver ORACLE JDBC KPRB es utilizado para escribir en Java procedimientos almacenados, funciones, triggers ... Ofrece acceso a la conexion activa por lo que no es necesario crear ninguna conexión adicional.

El driver JDBC KPRB de ORACLE permite obtener un handle de la conexion actual del contexto a través del método **OracleDriver.defaultConenction()**. Como hemos dicho, se utiliza para procedimientos almacenados, triggers .... Para ejecutar un procedimiento almacenado debemos estar conectados a ORACLE a través de una herramienta cliente como SQL*Plus., lo que hacemos con defaultConenction() es acceder a está conexion.

```
CREATE OR REPLACE AND COMPILE
JAVA SOURCE NAMED BANNER AS
import java.sql.*;
class dbAccess
{
  public static String showBanner() throws SQLException
  {
    String banner="";
    Connection conn =
    (new oracle.jdbc.driver.OracleDriver()).defaultConnection();
    Statement stmt = conn.createStatement();
    ResultSet rset =
              stmt.executeQuery("select BANNER from SYS.V_$VERSION");
    while (rset.next())
           banner += rset.getString(1);
    //stmt.close(); NO CERRAR LA CONEXION
    return banner;
  }
}
```

Es necesario crear un "wrapper" para poder ejecutar el programa desde PL/SQL o SQL.

```
CREATE OR REPLACE FUNCTION ShowBanner RETURN VARCHAR2
IS
LANGUAGE JAVA NAME 'dbAccess.showBanner() return java.lang.String';
```

Una vez hecho esto, podemos ejecutar la función como si se tratase de una función PL/SQL.

```
SELECT ShowBanner FROM DUAL;
```

Tomado de http://www.devjoker.com/contenidos/catss/132/Como-conectar-a-ORACLE-con-Java-.aspx